

Do AI Agents Really Improve Code Readability?

Kyogo Horikawa[†], Kosei Horikawa[§], Yutaro Kashiwa[§], Uwano Hidetake[†], Hajimu Iida[§]

[†]National Institute of Technology, Nara College, Japan

[§]Nara Institute of Science and Technology, Japan

AI1221@nara.kosen-ac.jp, uwano@info.nara-k.ac.jp

horikawa.kosei.hk1@naist.ac.jp, yutaro.kashiwa@is.naist.jp, iida@itc.naist.jp

Abstract

Code readability is fundamental to software quality and maintainability. Poor readability extends development time, increases bug-inducing risks, and contributes to technical debt. With the rapid advancement of Large Language Models, AI agent-based approaches have emerged as a promising paradigm for automated refactoring, capable of decomposing complex tasks through autonomous planning and execution. While prior studies have examined refactoring by AI agents, these analyses cover all forms of refactoring, including performance optimization and structural improvement. As a result, the extent to which AI agent-based refactoring specifically improves code readability remains unclear.

This study investigates the impact of AI agent-based refactoring on code readability. We extracted commits containing readability-related keywords from the AIDev dataset and analyzed changes in readability metrics before and after each commit, covering 403 commits evaluated using multiple quantitative metrics. Our results indicate that AI agents primarily target logic complexity (42.4%) and documentation improvements (24.2%) rather than surface-level aspects like naming conventions or formatting. However, contrary to expectations, readability-focused commits often degraded traditional quality metrics: the *Maintainability Index* decreased in 56.1% of commits, while *Cyclomatic Complexity* increased in 42.7%.

CCS Concepts

• **Software and its engineering** → **Integrated and visual development environments**; **Automatic programming**; *Software evolution*; *Maintaining software*.

Keywords

Agentic Coding, Coding Agent, Readability

ACM Reference Format:

Kyogo Horikawa[†], Kosei Horikawa[§], Yutaro Kashiwa[§], Uwano Hidetake[†], Hajimu Iida[§]. 2026. Do AI Agents Really Improve Code Readability?. In *23rd International Conference on Mining Software Repositories (MSR '26)*, April 13–14, 2026, Rio de Janeiro, Brazil. ACM, New York, NY, USA, 5 pages. <https://doi.org/10.1145/3793302.3793590>

1 Introduction

Code readability is a critical factor influencing software maintainability and sustainability [23]. Poor readability leads to technical

debt [5, 10] and increased communication costs [4, 26, 29]. To address this, software engineering research has proposed various approaches, including automated refactoring tools [13, 20, 32], quantitative metrics [3, 30], and machine learning-based prediction models [28, 31].

While these traditional approaches have reduced developer burden, the recent emergence of Large Language Models (LLMs) has elevated automated refactoring to a new level of practicality. Particularly since the emergence of conversational LLMs such as ChatGPT, numerous practical applications have been reported [12, 37]. In this workflow, developers provide refactoring instructions through prompts, and LLMs immediately present code improvement suggestions, achieving significant efficiency gains compared to conventional methods. Cases of LLM-assisted refactoring are actively shared in Stack Overflow and GitHub discussions [7, 36], indicating that this technology is becoming increasingly adopted in development practice.

More recently, there has been a shift from single prompt-based approaches to AI agent-based approaches. These approaches enable the decomposition of complex tasks into multiple subtasks, allowing for more sophisticated refactoring through autonomous cycles of planning, execution, and verification [38]. Existing research has analyzed what types of refactoring AI agents perform and to what extent [15]. However, these studies examine refactoring activities in general, which include not only readability-focused refactoring but also refactoring aimed at performance optimization [8], structural improvement [2], and other objectives [16]. As a result, the specific impact of AI agent-based refactoring on code readability has not been sufficiently clarified. This represents an important research gap, as understanding the effectiveness of agent-based approaches for readability improvement is essential for developers who seek to leverage these tools for this particular purpose.

Therefore, this study aims to clarify how much AI agent-based refactoring contributes to code readability. Specifically, we extracted commits containing keywords related to readability from the AIDev dataset [19] and investigated the impact of these commits on readability. We identified 403 commits and calculated three readability metrics for the code before and after each commit, allowing a thorough analysis of the effects of agent-based refactoring. Our findings show that readability-oriented commits do not necessarily lead to improvements in structural code quality and are often accompanied by quality degradation.

2 Related Work

Code Readability and Assessment. Code readability is central to maintainability, prompting extensive research into its quantification. Buse and Weimer [4] and Scalabrino *et al.* [29] proposed models to predict readability based on code features. Furthermore,



This work is licensed under a Creative Commons Attribution 4.0 International License. *MSR '26, Rio de Janeiro, Brazil*

© 2026 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-2474-9/2026/04

<https://doi.org/10.1145/3793302.3793590>

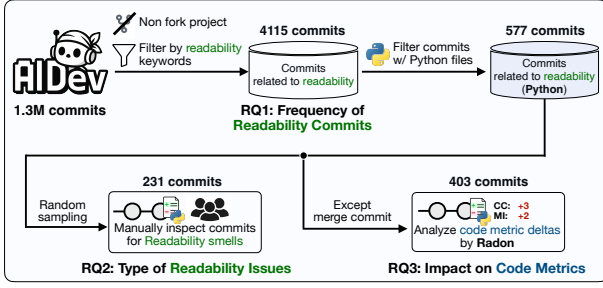


Figure 1: Overview of the study design

empirical studies by Dantas *et al.* [9] and Oliveira *et al.* [23] classified how human developers improve readability in practice, identifying categories such as identifier renaming and comment additions. More recently, Takerngsaksiri *et al.* [34] compared LLM-generated code with human-written code, finding comparable readability in enterprise settings. However, these studies focused on either human-driven improvements or code generation from scratch; whether AI agents can effectively improve the readability of existing code through refactoring has not been examined.

Refactoring by LLMs and Agents. The emergence of LLMs has advanced automated refactoring. While Cordeiro *et al.* [7] examined the refactoring capabilities of LLMs, their focus was on prompt-based approaches. More recently, autonomous coding agents have emerged [19, 38]. The closest work to ours is Horikawa *et al.* [15], who analyzed the impact of agent-driven refactoring on various code metrics. They did not observe any significant improvement in readability. However, they collected commits with commit messages containing “refactor”. This keyword is broad, and refactoring involves many operations including performance optimization [35], code restructuring [22], security enhancement [1, 11], and modularity improvement [40], beyond just readability. In this study, we focus on readability-improving commits made by agents and examine the impact on readability metrics established by Oliveira *et al.* [23].

3 Data Collection

In this section, we describe the data mining and data filtering as outlined in Figure 1.

3.1 Data Mining

To study the relationship between code readability and AI Agents, we start with the AIDev dataset [19]. This dataset contains 932,791 pull requests (PRs) created by five coding agents from over 61,000 repositories. However, the AIDev dataset only includes commit data for repositories with more than 100 GitHub stars. To gather a more comprehensive set of data points, we leverage the GitHub REST API¹ to mine all commits based on the provided metadata. In total, we collect 1,382,314 agentic commits.

3.2 Identifying Commits improving Readability

We then employ a keyword-based filtering approach on commit messages to extract commits related to readability improvement. Specifically, we performed case-insensitive filtering on commit

Table 1: Frequency of readability keywords

Keyword	Number of Commits (%)
readability	2,061 (50.1%)
clarity	2,003 (48.7%)
readable	254 (6.2%)
legibility	16 (0.4%)
easier to read	7 (0.2%)
understandable	3 (0.1%)
understandability	1 (0.0%)
comprehensible	0 (0.0%)

messages using eight keywords derived from prior work [6]: “readability”, “readable”, “understandability”, “understandable”, “clarity”, “legibility”, “easier to read”, and “comprehensible”. This set of keywords was selected to broadly capture different expressions referring to improvements in code readability and understandability. For RQ2 and RQ3, to ensure that the extracted commits correspond to source code readability improvements rather than documentation or configuration changes, we further filtered commits to those that include modifications to at least one Python source file.

4 Research Questions

RQ₁: How prevalent are readability-related commits generated by AI agents?

Motivation. Developers increasingly use AI agents for a wide range of software development tasks [19]. Prior work has observed that these agents frequently perform refactorings [38], and Horikawa *et al.* [15] examined how common such refactoring activities are. However, it is unclear to what extent these changes specifically target readability-improvements.

Approach. We flagged commits as readability-related if they contained at least one of these terms. We measure the frequency of readability-related commits and their percentage among all commits generated by AI agents (*i.e.*, all commits in the dataset).

Results. Table 1 shows the prevalence of readability-related commits, which explicitly describe code comprehensibility and readability. We found that 4,115 commits contained keywords related to readability, accounting for approximately 0.3% of all commits. On average, each selected commit contained 1.3 readability-related keywords. The most frequent term in commit messages generated by AI agents was “readability”, appearing in 50.1% of the commits.

These results indicate that, based on explicit commit messages, changes intended to improve readability account for a limited fraction of all AI-generated commits. Furthermore, AI agents tend to use standardized and general-purpose vocabulary when describing readability-related changes.

Answer to RQ₁. AI agents seldom focus on readability, with only 0.3% of agent-created commits related to it.

¹<https://docs.github.com/en/rest>

Table 2: Addressed Readability Smells: Human vs. Agent

Category	Human [23]	Agent
Complex, Long, or Inadequate Logic	18.2%	42.4%
Incomplete or Inadequate Code Doc.	22.3%	24.2%
Bad Identifier	20.3%	14.3%
Inconsistent or Disrupted Formatting	10.9%	14.3%
Inadequate Logging and Monitoring	4.7%	12.6%
Unnecessary Code	13.2%	10.0%
Wrong, Missing, or Inadequate String	8.1%	6.1%
Missing Constant Usage	2.3%	2.6%

RQ₂: What issues do agent-generated readability commits address?

Motivation. Prior research has identified several *readability smells* that impede code comprehension, including poor identifier naming, insufficient comments, and excessive logic complexity [23]. However, these studies focused on human-written code, and whether the same patterns appear in code generated or refactored by AI Agents has not been examined. Identifying the strengths and weaknesses of AI Agents in addressing *readability smells* would help developers better understand when to rely on these tools and where human review is still necessary.

Approach. From the 577 readability-related commits modifying Python files, we randomly sampled 231 commits to ensure a 95% confidence level with a 5% margin of error. To ensure classification reliability, we employed a closed card sorting approach [33]. Two authors, with 5 to 8 years of software development experience, independently inspected the commit messages and diff of the sampled commits. They mapped each commit to the eight *readability smells* defined by Oliveira *et al.* [23]. Since a commit may address multiple readability aspects, multi-labeling was permitted.

Following independent annotation, we evaluated inter-rater reliability. The initial classification achieved 94.7% label-level agreement (Micro-averaged Cohen’s Kappa: 0.80), with disagreements occurring on 98 of 1,832 labels. For these conflicting cases, a third inspector reviewed the labels and recommended resolutions. These recommendations were discussed among all three authors until complete consensus was reached.

Results. Table 2 compares the prevalence of *readability smells* addressed by AI agents versus human developers. Our analysis reveals distinct differences in prioritization between the two groups.

First, regarding AI agents, the results show a heavily skewed distribution. The top two categories, COMPLEX, LONG, OR INADEQUATE LOGIC and INCOMPLETE OR INADEQUATE CODE DOCUMENTATION, are the most dominant, collectively accounting for the majority of improvements. In contrast, MISSING CONSTANT USAGE and WRONG, MISSING, OR INADEQUATE STRING EXPRESSION OR LITERAL were the least addressed categories, representing the bottom two with negligible percentages (2.6% and 6.1%, respectively).

Second, a comparison with the baseline shows that agents exhibit a much stronger bias than human developers. As shown in the table, human contributions are relatively evenly distributed across several categories, such as INCOMPLETE OR INADEQUATE CODE DOCUMENTATION

(22.3%), BAD IDENTIFIER (20.3%), and COMPLEX, LONG, OR INADEQUATE LOGIC (18.2%). AI agents, however, prioritize logic complexity and documentation over surface-level readability aspects.

To illustrate this prioritization, we examine a representative improvement in COMPLEX, LONG, OR INADEQUATE LOGIC. In Listing 1, a complex string operation involving nested formatting and exception handling was simplified using `json.dumps`. This change abstracts away manual character escaping, making the logic clearer and easier to understand. This example illustrates that AI agents prioritize improvements that enhance understandability and maintainability over minor stylistic adjustments.

Listing 1: Example of Complex, Long, or Inadequate Logic²

```

1 yield rx.call_script(
2     f'''try {{ signals.identify('email.replace(chr(39), chr(92) + chr(39))) }} catch(e)
3     ) {{ console.warn('Signals identify failed:', e); }}'''
4     f'''try {{ signals.identify('json.dumps(email)) }} catch(e) {{ console.warn('
5     Signals identify failed:', e); }}'''

```

Answer to RQ₂. AI agents primarily focus on improving logic complexity and explanatory clarity rather than surface-level readability aspects.

RQ₃: How does code readability differ before and after AI agent-generated readability commits?

Motivation. Previous studies [23] indicate that developers spend approximately 70% of their time reading and understanding existing code, and they dedicate substantial maintenance effort to improving readability [18, 24]. Moreover, prior work has shown that such activities can lead to measurable improvements in code readability over time [24]. However, it is not clear to what extent AI agents improve readability in terms of quantitative metrics.

Approach. We extracted valid pre- and post-commit code states for 403 of 577 commits (excluding files with parsing errors). Using the radon library [27], we calculated the *Maintainability Index*, where higher values indicate better maintainability. *Maintainability Index* is a composite metric that combines static code measures such as lines of code, cyclomatic complexity, and Halstead metrics. We also calculated *Cyclomatic Complexity*, where lower values indicate simpler control structures. To measure code size at different levels of granularity, we recorded *Lines of Code* (total lines), *Logical Lines of Code*, *Source Lines of Code* (excluding blank lines and comments), and *Comment lines*. Additionally, we considered Halstead metrics, namely *Halstead Volume*, *Halstead Difficulty*, and *Halstead Effort*, to quantify different aspects of program complexity and cognitive effort. We chose radon due to its widespread adoption in prior empirical software engineering research [17, 21, 25].

We computed the change for each commit as $\Delta = \text{Metric}_{\text{after}} - \text{Metric}_{\text{before}}$. To determine statistical significance, we employed the Wilcoxon signed-rank test [39] ($p < 0.01$). We selected this non-parametric test because Q-Q plots exhibited S-shaped patterns, indicating that the paired differences did not follow a normal distribution. Additionally, we quantified the magnitude of the impact using Cliff’s delta [6] as the effect size.

Results. Table 3 summarizes the statistics of the metric differences between the pre- and post-commit code states. Overall, the p -values

²<https://github.com/reflex-dev/reflex-web/commit/68afd5c9e677e626218e7dc4b2584ff5c1e61b3e>

Table 3: Metric changes in readability-related commits

Metric	Mean	Median	<i>p</i> -value	ES
Lines of Code	27.61	6.00	<0.001	0.64
Cyclomatic Complexity	3.13	0.00	<0.001	0.63
Multi-line Comments	2.10	0.00	<0.001	0.63
Source Lines of Code	18.47	2.60	<0.001	0.60
Logical Lines of Code	12.18	1.00	<0.001	0.54
Halstead Volume	43.60	0.00	<0.001	0.50
Halstead Effort	321.49	0.00	<0.001	0.42
Halstead Difficulty	0.31	0.00	<0.001	0.35
Comment lines	1.67	0.00	<0.001	0.30
Single-line Comments	0.73	0.00	0.022	0.17
Maintainability Index	-3.25	-0.12	<0.001	-0.35

*ES: Effect Size

for most of the metrics are below 0.01, confirming significant differences before and after readability-related commits. Note that the Wilcoxon signed-rank test excludes pairs with zero differences. Consequently, even when the median difference is zero, the test can detect significant differences if the non-zero changes are predominantly in one direction.

The median increase in *Lines of Code* (6.00) is more than double that of *Source Lines of Code* (2.60), indicating that AI-based changes affected comments and formatting beyond executable statements, though the non-zero median for *Source Lines of Code* confirms that modifications to executable logic were also present. For Halstead metrics, the median difference for *Halstead Volume*, *Halstead Difficulty*, and *Halstead Effort* is zero, yet their mean values increased, implying that a specific subset of commits introduced substantial increases in code volume and cognitive effort.

As for effect sizes, *Maintainability Index* shows a medium effect size (-0.35), indicating a moderate shift in maintainability, while structural and size-related metrics such as *Cyclomatic Complexity* (0.63), *Lines of Code* (0.64), and *Source Lines of Code* (0.60) exhibit large effect sizes ($|r| \geq 0.5$), demonstrating strong and consistent increases in code size and complexity. Since a higher Maintainability Index is desirable while lower values are preferable for complexity and size metrics, these results suggest that AI-based readability-related commits actually degraded code quality from the perspective of traditional software metrics.

We also analyzed the proportions of improvement and deterioration across all commits. *Maintainability Index* deteriorated in 56.1% of the commits, indicating a tendency toward reduced maintainability. Similarly, *Lines of Code* increased in 71.5% of the cases, while *Cyclomatic Complexity* stayed unchanged in nearly half of the commits, with only 9.7% showing improvement. This reinforces that readability-oriented commits often involve increasing code volume without simplifying control structures.

Answer to RQ3. AI agent-generated readability commits do not necessarily improve structural code quality and often accompany increases in code volume and reduced maintainability.

5 Implication

Practitioners should critically review AI-proposed readability changes rather than accepting them automatically. Our findings show that AI agents rarely prioritize readability (0.3% in RQ1). Moreover, when they do, structural quality often degrades: 56.1% of commits worsened *Maintainability Index* and 42.7% increased *Cyclomatic Complexity* (RQ3). This suggests that current agents lack a robust understanding of effective readability improvements. **Researchers and developers should incorporate quality-checking mechanisms to prevent counterproductive refactoring.** Given this frequent structural degradation, tool developers should implement dedicated quality-checking agents. These agents can monitor metrics like Halstead complexity and code size (LOC) to reject refactorings that increase cognitive load. Future research should also enhance the verification phase, moving beyond functional correctness to include qualitative evaluations for better code organization.

6 Threats to Validity

Internal Validity: Manual classification in RQ2 introduces subjectivity. We mitigated this by having three independent inspectors classify commits and resolve disagreements through discussion.

Construct Validity: Our analysis may be affected by tangled commits, where multiple unrelated changes are bundled together, meaning observed metric changes may reflect modifications beyond readability improvements. Additionally, metrics such as MI and CC may not fully capture all aspects of readability, such as clarity of intent and logical flow.

External Validity: Our strict filtering resulted in a small sample size (0.3%), and our analysis focuses on Python projects, limiting generalizability to other languages. Future work should extend this study to additional languages and broader keyword sets.

7 Conclusion

This study investigated the impact of AI agent-based refactoring on code readability in open-source software projects. Our analysis of 403 readability-related PRs shows that AI agents primarily focus on reducing logic complexity and improving documentation, with less attention to surface-level aspects such as naming conventions and formatting. However, these changes do not necessarily improve structural quality metrics and are often accompanied by decreased maintainability and increased complexity.

Future research should focus on developing readability-focused agents that incorporate qualitative evaluations and consider long-term maintainability, drawing on human developers' improvement practices. Such agents would go beyond surface-level changes, such as renaming identifiers or adding comments, and instead prioritize modifications that fundamentally enhance code comprehensibility through better organization at the functional level.

Replication Package: The data and scripts used in our work is publicly available in the replication package [14].

Acknowledgments

This work was supported by JSPS KAKENHI (JP24K02921, JP25K21359) and JST PRESTO (JPMJPR22P3), ASPIRE (JPMJAP2415), and AIP Accelerated Program (JPMJCR25U7).

References

- [1] Abdullah Almogahed, Mazni Omar, and Nur Haryani Zakaria. 2022. Refactoring Codes to Improve Software Security Requirements. *Procedia Computer Science* 204 (2022), 108–115.
- [2] Bart Du Bois, Serge Demeyer, and Jan Verelst. 2004. Refactoring – Improving Coupling and Cohesion of Existing Code. In *Proc. of the 11th Working Conference on Reverse Engineering*. 144–151.
- [3] Raymond P. L. Buse and Westley Weimer. 2008. A metric for software readability. In *Proc. of the 2008 International Symposium on Software Testing and Analysis*. 121–130.
- [4] Raymond P. L. Buse and Westley Weimer. 2010. Learning a Metric for Code Readability. *IEEE Trans. Software Eng.* 36, 4 (2010), 546–558.
- [5] Shaiful Chowdhury, Hisham Kidwai, and Muhammad Asaduzzaman. 2025. Evidence is All We Need: Do Self-Admitted Technical Debts Impact Method-Level Maintenance?. In *Proc. of the 22nd International Conference on Mining Software Repositories*. IEEE, 813–825.
- [6] Norman Cliff. 1993. Dominance statistics: Ordinal analyses to answer ordinal questions. *Psychological Bulletin* 114, 3 (1993), 494–509.
- [7] Jonathan Cordeiro, Shayan Noei, and Ying Zou. 2024. An Empirical Study on the Code Refactoring Capability of Large Language Models. *CoRR* abs/2411.02320 (2024).
- [8] Vittorio Cortellessa, Jorge Andrés Díaz Pace, Daniele Di Pompeo, Sebastian Frank, Pooyan Jamshidi, Michele Tucci, and André van Hoorn. 2025. Introducing Interactions in Multi-Objective Optimization of Software Architectures. *ACM Trans. Softw. Eng. Methodol.* 34, 6 (2025), 181:1–181:39.
- [9] Carlos Eduardo de Carvalho Dantas, Adriano M. Rocha, and Marcelo de Almeida Maia. 2023. How do Developers Improve Code Readability? An Empirical Study of Pull Requests. In *Proc. of the 39th IEEE International Conference on Software Maintenance and Evolution*. 110–122.
- [10] Ella Dodor and Cristina V. Lopes. 2025. Checkstyle+: Reducing Technical Debt Through The Use of Linters with LLMs. *CoRR* abs/2510.23068 (2025).
- [11] Estonii Edward, Ally S. Nyamawe, and Noe Elisa. 2024. A Survey on Secure Refactoring. *SN Comput. Sci.* 5, 7 (2024), 952.
- [12] Angela Fan, Beliz Gokkaya, Mark Harman, Mitya Lyubarskiy, Shubho Sengupta, Shin Yoo, and Jie M. Zhang. 2023. Large Language Models for Software Engineering: Survey and Open Problems. In *Proc. of the 2023 IEEE/ACM International Conference on Software Engineering: Future of Software Engineering*. 31–53.
- [13] Yi Gao, Xing Hu, Xiaohu Yang, and Xin Xia. 2025. Automated Unit Test Refactoring. *ACM Softw. Eng.* 2 (2025), 713–733.
- [14] Kyogo Horikawa. 2025. Replication package. <https://github.com/horikawa-kyogo/replication-package/tree/main> Created: December 21th, 2025.
- [15] Kosei Horikawa, Hao Li, Yutaro Kashiwa, Bram Adams, Hajimu Iida, and Ahmed E. Hassan. 2025. Agentic Refactoring: An Empirical Study of AI Coding Agents. *CoRR* abs/2511.04824 (2025).
- [16] Martina Iammarino, Fiorella Zampetti, Lerina Aversano, and Massimiliano Di Penta. 2019. Self-Admitted Technical Debt Removal and Refactoring Actions: Co-Occurrence or More?. In *Proc. of the 2019 IEEE International Conference on Software Maintenance and Evolution*. IEEE, 186–190.
- [17] Oseremen Joy Idialu, Noble Saji Mathews, Rungroj Maipradit, Joanne M. Atlee, and Meiyappan Nagappan. 2024. Whodunit: Classifying Code as Human Authored or GPT-4 generated- A case study on CodeChef problems. In *Proc. of the 21st International Conference on Mining Software Repositories*. 394–406.
- [18] Miryung Kim, Thomas Zimmermann, and Nachiappan Nagappan. 2014. An Empirical Study of Refactoring Challenges and Benefits at Microsoft. *IEEE Transactions on Software Engineering (TSE)* 40, 7 (2014), 633–649.
- [19] Hao Li, Haoxiang Zhang, and Ahmed E. Hassan. 2025. The Rise of AI Team-mates in Software Engineering (SE) 3.0: How Autonomous Coding Agents Are Reshaping Software Engineering. *CoRR* abs/2507.15003 (2025).
- [20] Hao Liu, Yanlin Wang, Zhao Wei, Yong Xu, Juhong Wang, Hui Li, and Rongrong Ji. 2023. ReBERT: A Two-Stage Pre-trained Framework for Automatic Rename Refactoring. In *Proc. of 32nd ACM SIGSOFT International Symposium on Software Testing and Analysis*. 740–752.
- [21] Shuai Lu, Daya Guo, Shuo Ren, Junjie Huang, Alexey Svyatkovskiy, Ambrosio Blanco, Colin B. Clement, Dawn Drain, Daxin Jiang, Duyu Tang, Ge Li, Lidong Zhou, Linjun Shou, Long Zhou, Michele Tufano, Ming Gong, Ming Zhou, Nan Duan, Neel Sundaresan, Shao Kun Deng, Shengyu Fu, and Shujie Liu. 2021. CodeXGLUE: A Machine Learning Benchmark Dataset for Code Understanding and Generation. In *Proc. of the 35th Conference on Neural Information Processing Systems*.
- [22] Arjan J. Mooij, Jeroen Ketema, Steven Klusener, and Mathijs Schuts. 2020. Reducing Code Complexity through Code Refactoring and Model-Based Rejuvenation. In *Proc. of the 27th IEEE International Conference on Software Analysis, Evolution and Reengineering*. IEEE, 617–621.
- [23] Delano Oliveira, Reyndre Santos, Benedito de Oliveira, Martin Monperrus, Fernando Castor, and Fernanda Madeiral. 2025. Understanding Code Understandability Improvements in Code Reviews. *IEEE Trans. Software Eng.* 51, 1 (2025), 14–37.
- [24] Valentina Piantadosi, Fabiana Fierro, Simone Scalabrino, Alexander Serebrenik, and Rocco Oliveto. 2020. How does code readability change during software evolution? *Empir. Softw. Eng.* 25, 6 (2020), 5374–5412.
- [25] Russell A. Poldrack, Thomas Lu, and Gasper Begus. 2023. AI-assisted coding: Experiments with GPT-4. *CoRR* abs/2304.13187 (2023).
- [26] Daryl Posnett, Abram Hindle, and Premkumar T. Devanbu. 2011. A simpler model of software readability. In *Proc. of the 8th International Working Conference on Mining Software Repositories*. ACM, 73–82.
- [27] Radon. 2025. Welcome to Radon's documentation! <https://radon.readthedocs.io/en/latest/> Accessed: December 21th, 2025.
- [28] Devjeet Roy, Sarah Fakhoury, John Lee, and Venera Arnaudova. 2020. A Model to Detect Readability Improvements in Incremental Changes. In *Proc. of the 28th International Conference on Program Comprehension*. 25–36.
- [29] Simone Scalabrino, Gabriele Bavota, Christopher Vendome, Mario Linares-Vásquez, Denys Poshyvanyk, and Rocco Oliveto. 2021. Automatically Assessing Code Understandability. *IEEE Trans. Software Eng.* 47, 3 (2021), 595–613.
- [30] Simone Scalabrino, Mario Linares-Vásquez, Rocco Oliveto, and Denys Poshyvanyk. 2018. A comprehensive model for code readability. *J. Softw. Evol. Process.* 30, 6 (2018).
- [31] Simone Scalabrino, Mario Linares Vásquez, Denys Poshyvanyk, and Rocco Oliveto. 2016. Improving code readability models with textual features. In *Proc. of the 14th International Conference on Program Comprehension*. 1–10.
- [32] Mahnoosh Shahidi, Mehrdad Ashtiani, and Morteza Zakeri Nasrabadi. 2022. An automated extract method refactoring approach to correct the long method code smell. *J. Syst. Softw.* 187 (2022), 111221.
- [33] D. Spencer and J.J. Garrett. 2009. *Card Sorting: Designing Usable Categories*. Rosenfeld Media.
- [34] Wannita Takerngsaksiri, Chakkrit Tantithamthavorn, Micheal Fu, Jirat Pasuksmit, Kun Chen, and Ming Wu. 2025. Code Readability in the Age of Large Language Models: An Industrial Case Study from Atlassian. In *Proc. of the 41st IEEE International Conference on Software Maintenance and Evolution*. 732–742.
- [35] Luca Traini, Daniele Di Pompeo, Michele Tucci, Bin Lin, Simone Scalabrino, Gabriele Bavota, Michele Lanza, Rocco Oliveto, and Vittorio Cortellessa. 2022. How Software Refactoring Impacts Execution Time. *ACM Trans. Softw. Eng. Methodol.* 31, 2 (2022), 25:1–25:23.
- [36] Rosalia Tufano, Antonio Mastropaolo, Federica Pepe, Ozren Dabic, Massimiliano Di Penta, and Gabriele Bavota. 2024. Unveiling ChatGPT's Usage in Open Source Projects: A Mining-based Study. In *Proc. of the 21st International Working Conference on Mining Software Repositories*. 571–583.
- [37] Miku Watanabe, Yutaro Kashiwa, Bin Lin, Toshiki Hirao, Ken-ichi Yamaguchi, and Hajimu Iida. 2024. On the Use of ChatGPT for Code Review: Do Developers Like Reviews By ChatGPT?. In *Proc. of the 28th International Conference on Evaluation and Assessment in Software Engineering*. 375–380.
- [38] Miku Watanabe, Hao Li, Yutaro Kashiwa, Brittany Reid, Hajimu Iida, and Ahmed E. Hassan. 2025. On the Use of Agentic Coding: An Empirical Study of Pull Requests on GitHub. *CoRR* abs/2509.14745 (2025).
- [39] Frank Wilcoxon. 1945. Individual Comparisons by Ranking Methods. *Biometrics Bulletin* 1, 6 (1945), 80–83.
- [40] Liguo Yu and Srinivas Ramaswamy. 2008. Improving modularity by refactoring code clones: a feasibility study on Linux. *ACM SIGSOFT Softw. Eng. Notes* 33, 2 (2008).